MediKiosk System Documentation

Complete Technical, Architectural, and Datasets Reference Manual

All India Institute of Ayurveda — Smart India Hackathon

Date: 2026-09-16 | Commit: f118085510bdf2fa44785ee3ca5aaf5b4875c209

Project Analysis Summary

- **Last Analyzed**: 2026-09-16
- **Git Commit Hash**: `f118085510bdf2fa44785ee3ca5aaf5b4875c209`

1. Executive Summary & Purpose

MediKiosk is a touch- and voice-enabled clinical intake and triage kiosk platform developed for Indian hospital Outpatient Departments (OPDs), with specific clinical extensions for integrative medicine (Allopathy + AYUSH Dashavidha Pariksha).

The platform addresses high patient volume and language barriers in Indian public hospitals by allowing patients to self-record structured medical histories in English or Hindi before seeing a physician. It provides real-time emergency screening via deterministic red-flag rules, extracts structured data from uploaded prescriptions via OCR, and surfaces a real-time triage summary dashboard to attending clinicians with full data provenance tracking and FHIR R4 export capability.

2. Architecture Overview

MediKiosk follows a micro-service and layered client-server architecture:

1. **Frontend Kiosk & Doctor Portal** (`frontend/src/`): React 19 / 18 SPA built with Vite and TypeScript. Includes patient kiosk flows (`/welcome`, `/notice`, `/consent`, `/chief-complaint`, `/interview`, `/ayush-assessment`, `/records-upload`, `/preview`) and doctor triage dashboard (`/doctor/dashboard`, `/doctor/triage/:sessionId`). Supports IndexedDB offline buffering (`idb-keyval`). 2. **Backend Express API** (`backend/src/`): Express 5 server on Node.js 22. Manages session lifecycles, Zod schema validations, authentication (`bcryptjs`), real-time emergency evaluation (`redflags.ts`), AYUSH classical RAG reasoning (`ayushRag.ts`), and FHIR bundle generation (`fhir.ts`). 3. **AI & Speech Orchestration Router** (`backend/src/orchestration/router.ts`): Unified task router coordinating calls across: - **Sarvam AI**: Saaras v3 ASR, Bulbul v3 TTS, Doc AI digitise OCR, Mayura v1 Translation. - **Local MedGemma LLM**: `medgemma1.5:latest` via local Ollama instance for local red-flag and off-script Q&A. - **Google Gemini Cloud LLM**: Gemini 3.6 Flash / 3.5 Flash for RAG synthesis, ICD-10 suggestions, entity extraction, and fallback. 4. **Data & Vector Storage** (`Supabase PostgreSQL + pgvector`): Stores relational session tables (`sessions`, `answers`, `ayush_assessments`, `medical_history`, `documents`, `summaries`, `consent_logs`, `audit_logs`) and 768-dimensional embeddings in `rag_chunks` for RAG retrieval. Pushes live updates via Supabase WebSocket Realtime publications. 5. **Indic Speech Recognition Toolkit** (`models/polywhisper/`): Standalone PolyWhisper ASR system using frozen Whisper-Small with rank-16 per-language LoRA adapters for Hindi, Tamil, Telugu, Bengali, and Marathi, with INT8 ONNX graph export support.

3. Technology Stack Summary

Layer	Primary Technology	Auxiliary / Fallback Technology
Frontend UI	React 19 + Vite 8 + TypeScript	TailwindCSS + Lucide Icons + Radix UI Slot
Client Storage	IndexedDB via `idb-keyval`	Web Speech API / Web Speech Synthesis
Backend Framework	Node.js 22 + Express 5 + TypeScript	Zod + `bcryptjs` + CORS + Multer
Database	Supabase (PostgreSQL 15)	Realtime WebSockets (`supabase_realtime`)
Vector Search / RAG	Supabase `pgvector` (`rag_chunks`)	Cosine similarity scoring (`retriever.ts`)
Speech-to-Text (ASR)	Sarvam AI Saaras v3	PolyWhisper v9 / Web Speech API
Text-to-Speech (TTS)	Sarvam AI Bulbul v3	Web Speech Synthesis
Document OCR	Sarvam Doc AI digitise	Tesseract.js client-side WebAssembly
Local Clinical AI	MedGemma 1.5 via Ollama	Gemini 3.6 Flash / 3.5 Flash
Interoperability	HL7 FHIR R4 Bundle (`fhir.ts`)	Simulated ABDM M1/M2/M3 Services

4. End-to-End Workflow Summary

```
[Patient Kiosk Intake]
   ├── 1. DPDP Notice & Consent (/notice, /consent)
   ├── 2. Chief Complaint Selection (/chief-complaint)
   ├── 3. Adaptive Interview (/interview) -> Sarvam ASR / Voice-Match
   ├── 4. 15 Deterministic Red-Flag Rules (redflags.ts) -> Auto-Flag Emergencies
   ├── 5. 12-Dimension AYUSH Assessment -> Classical RAG Retrieval (ayushRag.ts)
   ├── 6. Prescription Document Upload -> OCR & Gemini Entity Extraction
   └── 7. Submission -> Generates Summary & Session Token
            |
            ▼ (Supabase Realtime WebSockets)
[Doctor Triage Dashboard]
   ├── 1. Live OPD Patient Queue with Red-Flag Indicators
   ├── 2. 9-Section Clinical Summary with Provenance Badges
   ├── 3. Inline Fact Verification (flips to 'doctor_entered')
   ├── 4. One-Click HL7 FHIR R4 Bundle Export
   └── 5. ABDM Care Context Linkage
```

5. Datasets Found in Repository

1. **OPD Clinical Dialogues Dataset** (`training/opd_dialogues.jsonl`): Synthetic Indian OPD doctor-patient clinical dialogues for QLoRA fine-tuning. 2. **Ayurveda / Traditional Medicine to ICD-10 Bridge Dataset** (`rag_corpus/raw/ayurveda_icd/`): 3,002 rows (`AmareshHebbar/ayurveda-icd-sft`) mapping Ayurvedic concepts to ICD-10 CM codes. 3. **Indian Vedas and Ayurveda Corpus** (`rag_corpus/raw/vedas/continueousPreTrainData.jsonl`): 39.51 MB continuous pre-training corpus containing Rig, Sama, Yajur, Atharva Veda, Charaka Samhita, Sushruta Samhita, Rasa Jala Nidhi, and IRJAY articles. 4. **IndicVoices-ST & FLEURS Speech Benchmarks** (`models/polywhisper/README.md`): Multilingual ASR training and evaluation speech datasets (`ai4bharat/indicvoices-st` and `google/fleurs`).

6. Key Project Strengths

1. **Robust Deterministic Safety Layer**: 15 deterministic red-flag emergency rules execute synchronously *before* AI model calls, ensuring critical emergencies (chest pain radiation, severe fever, stroke symptoms, suicidal ideation) are flagged without LLM hallucination risks. 2. **Data Provenance & Audit Trail**: Every clinical fact stores a `provenance` tag (`patient_reported` , `ocr_extracted` , `system_derived` , `doctor_entered`), ensuring full auditability during clinician review. 3. **Integrative Healthcare Support**: Blends Allopathy triage with AYUSH 12-dimension Dashavidha Pariksha, backed by vector RAG retrieval over classical texts. 4. **Multimodal Orchestration & Failover**: Architecture smoothly degrades from Sarvam AI -> MedGemma -> Gemini -> local rule-based fallbacks. 5. **Standards Interoperability**: Native FHIR R4 bundle generation and ABDM M1/M2/M3 specification compliance.

Technical References

- **Last Analyzed**: 2026-09-16
- **Git Commit Hash**: `f118085510bdf2fa44785ee3ca5aaf5b4875c209`

1. Regulatory, Data, and Interoperability Standards

1.1 HL7 FHIR R4 (Fast Healthcare Interoperability Resources)

- **Description**: Standard specification for electronic health record exchange. Used to map patient demographic, clinical complaint, AYUSH assessment, and consent data into an interoperable JSON bundle.
- **Why Relevant**: Enables seamless integration with hospital information systems (HIS) and National Health Authority (NHA) registries.
- **Where Used**: `backend/src/fhir.ts:L3-L157`, `backend/src/app.ts:L185-L193`.
- **Citation / Link**: HL7 FHIR Release 4 Standard

1.2 ABDM (Ayushman Bharat Digital Mission) Specs

- **Description**: Specifications for M1 (ABHA creation & Aadhaar OTP authentication), M2 (Care Context linking & FHIR transmission), and M3 (HIU consent flow).
- **Why Relevant**: Supports nationwide digital health ecosystem integration under NHA India.
- **Where Used**: `backend/src/services/abdm.ts:L1-L260`, `backend/src/routes/abdm.ts`.
- **Citation / Link**: ABDM Developer Portal

1.3 Digital Personal Data Protection (DPDP) Act 2023 & Right to Erasure

- **Description**: Indian data privacy framework mandating explicit patient consent logs and hard-deletion (Right to Erasure / consent revocation).
- **Why Relevant**: Ensures legal compliance for capturing health and demographic data in OPD kiosks.
- **Where Used**: `backend/src/app.ts:L250-L279`, `backend/src/routes/sessions.ts:L152-L163`.
- **Citation / Link**: DPDP Act 2023 Gazette Notification

1.4 CMS FY2026 ICD-10-CM Coding System

- **Description**: International Classification of Diseases, Tenth Revision, Clinical Modification used to code clinical diagnoses and traditional medicine bridges.
- **Why Relevant**: Maps clinical summaries and traditional Ayurvedic conditions to standardized billing/diagnostic codes.
- **Where Used**: `rag_corpus/raw/ayurveda_icd/README.md:L60-L64`, `backend/src/summary.ts:L153-L174`.
- **Citation / Link**: CMS ICD-10-CM Official Guidelines

2. Artificial Intelligence & Machine Learning Architectures

2.1 Google Gemini LLM API (Gemini 3.6 Flash / 3.5 Flash)

- **Description**: Multimodal foundation model for RAG reasoning, document entity extraction, clinical summary generation, ICD-10 coding suggestions, and Hindi clinical translation.
- **Why Relevant**: Primary cloud LLM provider powering semantic triage and clinical summary generation.
- **Where Used**: `backend/src/app.ts:L158-L160`, `backend/src/rag/ayushRag.ts:L165-L215`, `backend/src/summary.ts:L136-L174`, `backend/src/clinicalAnalysis.ts:L12-L55`.
- **Citation / Link**: Google Gemini API Documentation

2.2 MedGemma 1.5 (via Ollama Local Runtime)

- **Description**: Domain-adapted medical LLM (`medgemma1.5:latest`) executed via Ollama on local hardware.
- **Why Relevant**: Primary offline/local LLM for semantic red-flag detection and off-script Q&A before falling back to Gemini.
- **Where Used**: `backend/src/orchestration/medgemmaClient.ts:L1-L24`, `backend/src/services/ollamaBootstrap.ts:L1-L56`.
- **Citation / Link**: TODO: verify official paper citation for MedGemma 1.5.

2.3 Sarvam AI API Suite

- **Description**: Sovereign Indic AI models:

  - **Saaras v3**: Multilingual Speech-to-Text (ASR) supporting Hindi, Tamil, Telugu, Bengali, Marathi, etc. - **Bulbul v3**: Multilingual Text-to-Speech (TTS). - **Doc AI Job Digitise**: Document OCR & structure parsing. - **Mayura v1**: Translation model between Indic languages and English.
- **Why Relevant**: Primary server-side Indic speech recognition, voice synthesis, document digitization, and translation.
- **Where Used**: `backend/src/orchestration/sarvamClient.ts:L1-L119`, `backend/src/orchestration/router.ts:L22-L58`.
- **Citation / Link**: Sarvam AI Developer Documentation

2.4 Tesseract.js (v7.0.0)

- **Description**: Pure JavaScript OCR engine based on Tesseract C++ engine compiled to WebAssembly.

- **Why Relevant**: Client-side and offline fallback OCR engine for reading medical prescription images.
- **Where Used**: `backend/src/app.ts:L15`, `backend/src/app.ts:L627`, `frontend/package.json:L25`.
- **Citation / Link**: Tesseract.js GitHub Repository

3. Traditional Ayurvedic Literature & Clinical Frameworks

3.1 Dashavidha Pariksha (10-Fold Clinical Assessment)

- **Description**: Classical Ayurvedic diagnostic framework evaluating Prakriti, Vikriti, Agni, Koshtha, Sara, Samhanana, Pramana, Satmya, Sattva, Ahara Shakti, Vyayama Shakti, and Vaya.
- **Why Relevant**: Forms the core structure for the 12-dimension AYUSH intake questionnaire and deterministic reasoning module.
- **Where Used**: `backend/src/ayushReasoning.ts:L1-L150`, `backend/src/rag/ayushRag.ts:L114-L126`.
- **Source**: Charaka Samhita, Vimanasthana Chapter 8 (Rogabhishagjitiya Vimana).

3.2 Indian Vedas and Classical Ayurveda Corpus

- **Description**: Comprehensive text corpus comprising Rig Veda, Sama Veda, Yajur Veda, Atharva Veda, Charaka Samhita, Sushruta Samhita, Rasa Jala Nidhi, and IRJAY research articles.
- **Why Relevant**: Source raw materials indexed into vector storage for RAG retrieval during Ayush clinical reasoning.
- **Where Used**: `rag_corpus/raw/vedas/README.md:L1-L151`, `backend/src/rag/retriever.ts:L78-L86`.
- **Citation / Link**:

CODE

```bibtex
  @misc{vedas_ayurveda_corpus_2026,
    title = {Indian Vedas and Ayurveda Corpus},
    year = {2026},
    publisher = {HuggingFace}
  }
```

System Workflow & Execution Architecture

- **Last Analyzed**: 2026-09-16
- **Git Commit Hash**: `f118085510bdf2fa44785ee3ca5aaf5b4875c209`

---

1. End-to-End System Architecture & Data Flow

```
flowchart TD
    subgraph Data Ingestion & Preprocessing
        A1["Raw Vedas & Ayurveda Texts
(rag_corpus/raw/vedas)"] --> A2["Corpus Parser
(backend/src/rag/parse.ts)"]
        A2 --> A3["Semantic Chunking
(backend/src/rag/chunk.ts)"]
        A3 --> A4["Gemini Embeddings Generator
(backend/src/rag/embed.ts)"]
        A4 --> A5["Database Pre-Seeder
(backend/src/rag/preseed.ts)"]
        A5 --> A6[("Supabase pgvector
rag_chunks table")]
    end

subgraph Fine-Tuning & Model Prep
        B1["OPD Dialogues
(training/opd_dialogues.jsonl)"] --> B2["QLoRA Fine-Tuner
(training/qlora_finetune.py)"]
        B2 --> B3["Tuned Medra/Llama-3 Checkpoint
(medikiosk-model/)"]
        B4["IndicVoices ST Audio"] --> B5["PolyWhisper LoRA Training
(models/polywhisper)"]
        B5 --> B6["ONNX INT8 Quantized Graphs
(models/polywhisper/export)"]
    end



subgraph Patient Kiosk Intake Pipeline
        C1["Patient at Kiosk Web App
(frontend/src/pages)"] --> C2["DPDP Consent & Notice
(/notice, /consent)"]
        C2 --> C3["Chief Complaint Selection
(/chief-complaint)"]
        C3 --> C4["Adaptive Interview Flow
(/interview)"]

        C4 -->|Voice Audio| C5["Sarvam Saaras v3 ASR
(/api/sarvam/asr)"]
        C4 -->|Text Selection| C6["Zod Schema Validation
(backend/src/schemas)"]

        C4 --> C7["15 Deterministic Red-Flag Rules
(backend/src/redflags.ts)"]
        C7 -->|Emergency Flagged| C8["Mark Session Red-Flag
(red_flags table)"]



C4 --> C9["AYUSH Dashavidha Assessment
(/ayush-assessment)"]
        C9 --> C10["RAG Ayush Reasoning Engine
(backend/src/rag/ayushRag.ts)"]
        A6 -->|Vector Similarity Search| C10



C4 --> C11["Document OCR Intake
(/records-upload)"]
        C11 -->|Image Upload| C12["Sarvam Doc AI / Tesseract
(/api/sarvam/ocr)"]
        C12 --> C13["Gemini Clinical Entity Extractor
(/api/gemini/extract-entities)"]



C10 --> C14["Submit & Token Issuance
(/preview, /submitted)"]
        C13 --> C14
        C14 --> C15[("Supabase PostgreSQL DB
sessions, answers, summaries")]
    end
```

```
subgraph Doctor Triage & Clinical Dashboard
        C15 -->|Realtime WebSocket Push| D1["Doctor Triage Queue
(frontend/src/pages/Doctor)"]
        D1 --> D2["Provenance-Badged Summary
(patient_reported vs doctor_entered)"]
        D2 --> D3["Inline Verification & Editing
(PATCH /history-facts/:id)"]
        D3 --> D4["FHIR R4 Bundle Export
(GET /api/fhir/:sessionId)"]
        D4 --> D5["ABDM Care Context Link
(POST /api/abdm/link-care-context)"]
    end
```

2. Setup and Installation Workflow

Prerequisites

- **Node.js**: v22 LTS (specified in `.node-version`, `package.json`, `Dockerfile`)
- **npm**: ^10.0.0
- **Docker & Docker Compose** (optional for containerized runtime)
- **Ollama Runtime** (optional for local MedGemma LLM execution)

Execution Commands

BASH

```
npm install
cd backend && npm install
cd ../frontend && npm install
```

```
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env
```

3. Data Ingestion & Preprocessing Workflow

- **Entry Point**: `backend/src/rag/build_full_corpus.ts` & `backend/src/rag/preseed.ts`.
- **Config Files**: `backend/.env` (`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `GEMINI_API_KEY`).
- **Inputs**: Raw Vedic and Ayurvedic text datasets in `rag_corpus/raw/`.
- **Outputs**: Embedded chunks stored in Supabase table `rag_chunks`.

Order of Execution

```bash
npx tsx backend/src/rag/parse.ts
npx tsx backend/src/rag/chunk.ts
npx tsx backend/src/rag/embed.ts
npx tsx backend/src/rag/preseed.ts
```

4. Model Training & Fine-Tuning Workflow

4.1 OPD Clinical Dialogue QLoRA Fine-Tuning

- **Entry Point**: `training/qlora_finetune.py`.
- **Base Model**: `meta-llama/Meta-Llama-3-8B-Instruct`.
- **Dataset Input**: `training/opd_dialogues.jsonl`.
- **Hyperparameters**:
  - Quantization: 4-bit NormalFloat (`nf4`) double quantization, `bfloat16` compute. - LoRA Config: rank `r=16` , `lora_alpha=32` , `target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"]` . - Batch size: 4 per device, gradient accumulation steps: 4. - Learning rate: `2e-4` with cosine scheduler.
- **Output Artifact**: `./medikiosk-model/` PEFT adapter weights.
- **Execution Command**:

```bash
python training/qlora_finetune.py
```

4.2 PolyWhisper Multilingual Indic ASR Training & ONNX Export

- **Entry Point**: `models/polywhisper/app.py`.
- **Base Model**: `openai/whisper-small` (244M params).
- **Adapters**: Rank-16 LoRA per language (Hindi, Tamil, Telugu, Bengali, Marathi) (~14MB each).
- **ONNX Export Command**:

```bash
polywhisper export --lang hi --variant prod --int8
```

5. Runtime Inference & Prediction Pipeline

5.1 Patient Intake & Real-Time Emergency Screening

1. **Patient Authentication & Consent**: Patient opens Kiosk UI (`/welcome`), reviews DPDP Notice (`/notice`), and grants consent (`/consent`). 2. **Chief Complaint & Adaptive Interview**: Patient records symptoms via speech (`/api/sarvam/asr`) or touch UI. 3. **Deterministic Safety Evaluation**: Each submitted answer is evaluated by 15 deterministic red-flag rules (`backend/src/redflags.ts:L4-L20`). If matched, session `red_flag` flips to `true` and records entry in `red_flags` table. 4. **Secondary Semantic Safety**: Answers without deterministic matches undergo secondary evaluation via MedGemma LLM / Gemini (`backend/src/redflags.ts:L69-L99`). 5. **AYUSH RAG Assessment**: AYUSH 12-dimension responses are evaluated by `reasonAyushWithRag()` (`backend/src/rag/ayushRag.ts:L55-L277`). Vector similarity query retrieves top-5 classical text chunks from Supabase `rag_chunks` to generate cited clinical

insights. 6. **Prescription OCR Extraction**: Uploaded prescription images are processed via Sarvam Doc AI digitise or local Tesseract.js (`backend/src/app.ts:L976-L1010`), followed by Gemini clinical entity extraction (`backend/src/app.ts:L1012-L1068`).

5.2 Doctor Triage & Clinical Verification

1. **Live OPD Queue**: Clinician logs in (`/auth/login`) and views active OPD sessions sorted by triage priority and red-flag status. Real-time updates pushed via Supabase WebSocket publication `supabase_realtime` . 2. **Inline Verification**: Doctor reviews provenance tags (`patient_reported` , `ocr_extracted` , `system_derived`). Edits or verifies facts, which updates provenance to `doctor_entered` (`PATCH /history-facts/:id`). 3. **FHIR R4 Bundle Export**: Clinician exports complete clinical visit record as an HL7 FHIR R4 Bundle (`GET /api/fhir/:sessionId`). 4. **ABDM Linkage**: Care Context linked with ABHA address (`POST /api/abdm/link-care-context`).

6. Deployment & Monitoring Workflow

Local Development / Demo Mode

BASH

```
npm run demo
```

```
npm run demo:stop
```

Docker Containerized Deployment

BASH

```
docker compose up -d
```

Cloud Production Targets

- **Backend**: Deployed to Railway via Nixpacks Node.js 22 runtime (`railway.json`).
- **Frontend**: Deployed to Vercel with SPA rewrite rules (`vercel.json`).

Monitoring & Event Auditing

- Real-time orchestration event stream exposed via SSE at `GET /api/orchestration/events` (`backend/src/app.ts:L109-L124`).
- All doctor verifications, data access, and consent revocations written to `audit_logs` table (`backend/src/app.ts:L265` , `backend/src/app.ts:L764`).

Tools and Technologies Inventory

- **Last Analyzed**: 2026-09-16
- **Git Commit Hash**: `f118085510bdf2fa44785ee3ca5aaf5b4875c209`

Technical Inventory Table

Category	Tool / Technology	Version	Purpose	Where Used (File Paths)	Evidence / Lockfile Reference
Programming Languages	TypeScript	`^5.5.4` (backend) `~6.0.2` (frontend)	Primary strongly-typed language for backend and frontend codebase	`backend/package.json:L48`, `frontend/package.json:L42`	`package.json` dependenci
Programming Languages	JavaScript (Node.js ES Modules & CommonJS)	Node `22`	Server runtime, build scripts, demo automation scripts	`.node-version:L1`, `backend/Dockerfile:L2`, `railway.json:L9`	`.node-version`, `Dockerf`
Programming Languages	Python	`3.9+`	Fine-tuning scripts, ASR model demos, evaluation routines	`models/polywhisper/README.md:L103`, `training/qlora_finetune.py`	`models/polywhisper/app.` `qlora_finetune.py`
Programming Languages	SQL (PostgreSQL Dialect)	PostgreSQL 15+	Relational database schema, vector extensions, and migration scripts	`backend/schema.sql:L1-L174`, `survival_migration.sql`	`schema.sql`, `survival_m`
Frontend Frameworks	React	`^19.2.8`	Component-based user interface framework for patient kiosk & doctor portal	`frontend/package.json:L20`	`frontend/package.json`
Frontend Frameworks	React DOM	`^19.2.8`	Virtual DOM renderer for React web application	`frontend/package.json:L21`	`frontend/package.json`
Routing	React Router DOM	`^7.18.3`	Client-side routing across kiosk screens and doctor dashboard	`frontend/package.json:L22`	`frontend/package.json`
Build Tools	Vite	`^8.2.2`	Next-generation frontend bundler and local development server	`frontend/package.json:L43`, `frontend/vite.config.ts`	`frontend/package.json`
Backend Framework	Express	`^5.2.1`	HTTP web server framework for REST endpoints and middleware	`backend/package.json:L23`, `backend/src/app.ts:L26`	`backend/package.json`
Backend Execution	`tsx`	`^4.23.13`	TypeScript execute and watch runtime for Node.js	`backend/package.json:L7`, `backend/package.json:L47`	`backend/package.json`
Styling & Design System	Tailwind CSS	`^3.4.19`	Utility-first CSS framework for Ayurvedic theme styling	`frontend/package.json:L41`, `frontend/tailwind.config.ts`	`frontend/package.json`
Styling Utilities	`clsx` & `tailwind-merge`	`^2.1.1` / `^3.6.0`	Conditional class composition and Tailwind rule merging	`frontend/package.json:L17`, `frontend/package.json:L23`	`frontend/package.json`
Iconography	Lucide React	`^1.39.0`	Modern SVG icon set for UI components	`frontend/package.json:L19`	`frontend/package.json`
UI Primitives	Radix UI Slot	`^1.3.3`	Unstyled accessible UI primitive slots for component wrapping	`frontend/package.json:L14`	`frontend/package.json`
Database & Realtime	Supabase JS Client	`^2.114.0`	PostgreSQL ORM client, auth verification, and WebSocket Realtime subscriptions	`backend/package.json:L18`, `frontend/package.json:L15`, `backend/src/supabase.ts`	`package.json` lockfiles
Database Extensions	Supabase `pgvector`	`0.5.0+` (TODO: verify)	Vector similarity search for RAG embeddings in `rag_chunks` table	`backend/src/rag/retriever.ts:L39-L115`	`backend/src/rag/preseed`

Category	Tool / Technology	Version	Purpose	Where Used (File Paths)	Evidence / Lockfile Reference
Client-Side Storage	`idb-keyval`	`^6.3.0`	Lightweight IndexedDB key-value wrapper for offline patient answer buffering	`frontend/package.json:L18`	`frontend/package.json`
Cloud AI SDK	Google Generative AI	`^0.24.1`	Client library for Gemini 3.6 Flash / 3.5 Flash LLM synthesis and embeddings	`backend/package.json:L17`, `backend/src/app.ts:L153`	`backend/package.json`
Speech & Vision API	Sarvam AI API	`v3` / `v1`	External REST API for Saaras v3 ASR, Bulbul v3 TTS, Doc AI digitise OCR, Mayura v1 Translation	`backend/src/orchestration/sarvamClient.ts:L1-L119`	`backend/src/orchestrati`
Local LLM Runtime	Ollama	`latest` (TODO: verify)	Background service manager running `medgemma1.5:latest` for local inference	`backend/src/services/ollamaBootstrap.ts:L1-L56`	`backend/src/services/ol`
Client & Fallback OCR	Tesseract.js	`^7.0.0`	Pure JS / WebAssembly OCR engine for prescription text extraction	`backend/package.json:L29`, `frontend/package.json:L25`	`package.json` files
Python ML Frameworks	PyTorch	`2.0+` (TODO: verify)	Deep learning framework for PolyWhisper and QLoRA fine-tuning	`models/polywhisper/README.md:L104`, `training/qlora_finetune.py:L8`	`training/qlora_finetune`
Python ML Libraries	Hugging Face `transformers`, `peft`, `trl`, `datasets`, `bitsandbytes`	TODO: verify	Model loading, LoRA adapter injection, 4-bit quantization, SFT Trainer	`training/qlora_finetune.py:L9-L17`	`training/qlora_finetune`
Python UI & Edge	Gradio	`^4.0.0` (TODO: verify)	Interactive Web UI demo for PolyWhisper Indic ASR	`models/polywhisper/app.py:L14`	`models/polywhisper/app.`
Python Runtime / ONNX	ONNX Runtime	TODO: verify	CPU-optimized INT8 quantized graph inference for PolyWhisper ASR	`models/polywhisper/README.md:L105`	`models/polywhisper/READ`
Authentication & Security	`bcryptjs`	`^3.0.3`	Salted password hashing (10 rounds) for doctor accounts	`backend/package.json:L20`, `backend/src/app.ts:L5`	`backend/package.json`
Security Middleware	`cors` & `helmet` security headers	`^2.8.6`	Cross-Origin Resource Sharing control and HTTP header hardening	`backend/package.json:L21`, `backend/src/middleware/security.ts`	`backend/package.json`
Payload Validation	Zod	`^4.6.4`	TypeScript-first schema validation for API request bodies	`backend/package.json:L31`, `backend/src/schemas/index.ts`	`backend/package.json`
File Upload Handling	Multer & Form-Data	`^2.3.0` / `^4.0.6`	Middleware for handling `multipart/form-data` audio/document uploads	`backend/package.json:L24-L25`, `backend/src/app.ts:L157`	`backend/package.json`
E2E Browser Testing	Playwright	`^1.63.0` (frontend) `^1.62.1` (backend)	End-to-end user journey, voice recognition, and real-time verification testing	`frontend/package.json:L29`, `backend/package.json:L27`, `frontend/playwright.config.ts`	`package.json` files
Unit & Integration Testing	Vitest & Jest	`^4.1.11` (vitest)	Frontend unit runner (Vitest) and backend API	`frontend/package.json:L44`, `backend/package.json:L42`	`package.json` files

Category	Tool / Technology	Version	Purpose	Where Used (File Paths)	Evidence / Lockfile Reference
		`^30.5.1` (jest)	route runner (Jest + Supertest)		
Accessibility Testing	`@axe-core/playwright`	`^4.13.0`	Automated WCAG accessibility auditing in Playwright tests	`frontend/package.json:L28`	`frontend/package.json`
Linting & Quality	Oxlint	`^1.79.0`	High-performance Rust-based JavaScript/TypeScript linter	`frontend/package.json:L39`, `frontend/.oxlintrc.json`	`frontend/package.json`
Containerization	Docker & Docker Compose	Docker API v2	Multi-stage Docker building Node 22 Alpine and Nginx Alpine containers	`Dockerfile` (backend), `Dockerfile` (frontend), `docker-compose.yml`	`docker-compose.yml`
Cloud Deployment	Railway & Vercel	Nixpacks / Vite	Cloud hosting configuration manifests for backend API and frontend static assets	`railway.json`, `vercel.json`, `frontend/vercel.json`	Configuration JSON files
CLI Automation	Concurrently	`^9.0.0`	Running backend and frontend dev servers in parallel during local demo	`package.json:L15`	Root `package.json`

Datasets Documentation

- **Last Analyzed**: 2026-09-16
- **Git Commit Hash**: `f118085510bdf2fa44785ee3ca5aaf5b4875c209`

Dataset Inventory Summary

The repository includes, processes, or references four distinct dataset collections covering clinical OPD dialogues, traditional Ayurvedic text collections, ICD-10 medical code mappings, and multilingual ASR benchmarks.

1. OPD Clinical Dialogues Dataset (Synthetic)

- **Name**: Indian OPD Clinical Intake & Triage Dialogues
- **Type**: Synthetic Text (JSONL format)
- **Source / URL**: Local repo file `training/opd_dialogues.jsonl`
- **Version / Hash**: TODO: verify commit snapshot
- **License**: TODO: verify repository license (MIT)
- **Size**: 1.46 KB (Sample demonstration set)
- **Format**: JSON Lines (`.jsonl`)
- **Local Path**: `training/opd_dialogues.jsonl`
- **Download / Access Instructions**: Included directly in repository.
- **Preprocessing Steps**: Formatted into Llama 3 ChatML / instruction prompt structure via `format_prompt()` in `training/qlora_finetune.py:L58-L69`.
- **Train / Validation / Test Splits**:
  - Train: 100% (Demonstration split loaded via `load_dataset("json", data_files="opd_dialogues.jsonl", split="train")`) - Validation / Test: None
- **Columns / Features / Labels**:
  - `system`: System instruction defining clinical role & triage guidelines. - `user`: Patient chief complaint and symptom inputs. - `assistant`: Structured clinical assistant triage response.
- **Target Variable**: `assistant` text (causal language modeling target).
- **Where Used in Code**: `training/qlora_finetune.py:L21`, `training/qlora_finetune.py:L77`.
- **Model / Task it Supports**: Meta-Llama-3-8B QLoRA fine-tuning for Indian hospital OPD clinical history taking.
- **Citation**: Synthetic dataset generated for hackathon demonstration.
- **Privacy / Ethical Notes**: 100% synthetic clinical data; contains zero real patient Personally Identifiable Information (PII) or Protected Health Information (PHI).

2. Ayurveda / Traditional Medicine to ICD-10 Bridge Dataset

- **Name**: `AmareshHebbar/ayurveda-icd-sft` (AxisMapper Medical AI Suite)
- **Type**: Text (JSONL format)
- **Source / URL**: Hugging Face Dataset - AmareshHebbar/ayurveda-icd-sft
- **Version / Hash**: CMS FY2026 ICD-10-CM release baseline
- **License**: Apache 2.0 (`rag_corpus/raw/ayurveda_icd/README.md:L2`)
- **Size**: 1.64 MB total across splits (3,002 rows)
- **Format**: JSON Lines (`.jsonl`)
- **Local Path**: `rag_corpus/raw/ayurveda_icd/`
  - `train.jsonl` (1.31 MB) - `validation.jsonl` (162 KB) - `test.jsonl` (165 KB)
- **Download / Access Instructions**:

```python
from datasets import load_dataset
  ds = load_dataset("AmareshHebbar/ayurveda-icd-sft")
```

- **Preprocessing Steps**: Extracted from public CMS ICD-10-CM Z-codes (health status) and Y-codes (external causes) mapped to traditional Indian medicine terms (Ayurveda, Unani, Siddha, Homeopathy). Formatted as ChatML `messages` list.
- **Train / Validation / Test Splits**:
  - Train: 2,401 rows - Validation: 300 rows - Test: 301 rows - Total: 3,002 rows
- **Columns / Features / Labels**:
  - `messages`: List of chat objects `[{"role": "system", "content": "..."}, {"role": "user", "content": "..."}, {"role": "assistant", "content": "..."}]`

- **Target Variable**: Assistant message containing ICD-10-CM code, description, and clinical context notes.
- **Where Used in Code**: Parsed by `backend/src/rag/parse.ts` and indexed into vector storage for RAG retrieval during Ayush clinical reasoning.
- **Model / Task it Supports**: Integrative medicine clinical documentation, ABDM ABHA health records integration, and ICD-10 automated coding.
- **Citation**:

```
bibtex
  @misc{axiomapper2026,
    author    = {Hebbar, Amaresh},
    title     = {AxisMapper: Medical AI Fine-tuning Dataset Suite},
    year      = {2026},
    publisher = {HuggingFace},
    url       = {https://huggingface.co/collections/AmareshHebbar/axiomapper-medical-ai-suite}
  }
```

- **Privacy / Ethical Notes**: Extracted strictly from public CMS guidelines and medical coding reference standards; no personal health records used.

---

3. Indian Vedas and Ayurveda Corpus

- **Name**: Indian Vedas and Ayurveda Continuous Pre-Training Corpus
- **Type**: Multilingual Text (Sanskrit `sa`, English `en`, Hindi `hi`)
- **Source / URL**: Multi-source digital humanities collections (Griffith, Keith, Gulabkunverba Ayurvedic Society, Bhishagratna, Mookerji, IRJAY journal articles)
- **Version / Hash**: Public domain digital translations
- **License**: Public Domain / Open Educational (`license_name: public-domain` in `rag_corpus/raw/vedas/README.md:L3`)
- **Size**: 39.51 MB (`continueousPreTrainData.jsonl`)
- **Format**: Single unified JSON Lines file (`.jsonl`)
- **Local Path**: `rag_corpus/raw/vedas/continueousPreTrainData.jsonl`
- **Download / Access Instructions**: Included in local repo directory.
- **Preprocessing & Chunking Strategy**:

  - **Vedic Texts (Rig, Sama, Yajur, Atharva Veda)**: Chunked at Hymn (Sukta), Anuvaka, or Verse granularity. - **Ayurvedic Samhitas (Charaka, Sushruta, Rasa Jala Nidhi)**: Chunked at Chapter / Section level. - **IRJAY Research Papers**: Page-level chunking with 150-character overlap.
- **Train / Validation / Test Splits**: Single continuous pre-training / RAG retrieval split (~10K to ~100K records).
- **Columns / Features / Labels**:

  - `text`: Structured payload formatted with collection title, translator, chapter/sutra heading, and translation text payload.
- **Target Variable**: Unsupervised text / RAG context retrieval payload.
- **Where Used in Code**: Parsed by `backend/src/rag/parse.ts`, chunked in `backend/src/rag/chunk.ts`, embedded via Gemini API (`embed.ts`), and queried by `backend/src/rag/retriever.ts`.
- **Model / Task it Supports**: Classical Ayush RAG reasoning engine (`ayushRag.ts`), VedaGPT domain pre-training.
- **Citation**: Rig Veda (Ralph T.H. Griffith 1896), Black Yajur Veda (Arthur Berriedale Keith 1914), Sushruta Samhita (Kaviraj Kunja Lal Bhishagratna 1907), Rasa Jala Nidhi (Rasacharya Kaviraj Bhudeb Mookerji 1926).
- **Privacy / Ethical Notes**: Ancient historical and classical literature; no ethical or privacy constraints.

---

4. IndicVoices-ST & FLEURS Speech Datasets (PolyWhisper ASR)

- **Name**: `ai4bharat/indicvoices-st` (Training) & `google/fleurs` (Evaluation)
- **Type**: Audio Speech (.wav) & Transcripts (.txt)
- **Source / URL**:

  - Hugging Face Datasets - ai4bharat/indicvoices-st - Hugging Face Datasets - google/fleurs
- **Version / Hash**: FLEURS Benchmark release
- **License**: Creative Commons Attribution (CC-BY)
- **Size**: ~19k–20k clips per language for IndicVoices-ST
- **Format**: Audio 16kHz WAV + UTF-8 transcript strings
- **Local Path**: External datasets loaded during model training (eval evaluation outputs saved in `models/polywhisper/eval_*.json`)
- **Download / Access Instructions**:

```
PYTHON
from datasets import load_dataset
  fleurs_hi = load_dataset("google/fleurs", "hi_in")
  indicvoices_hi = load_dataset("ai4bharat/indicvoices-st", "hi")
```

- **Preprocessing Steps**: SpecAugment time/frequency masking + 0.9x/1.1x speed perturbation applied selectively to hard languages (Bengali `bn`, Marathi `mr`), clean training for `hi` / `ta`.
- **Train / Validation / Test Splits**: Standard FLEURS train/dev/test splits for Hindi (`hi_in`), Tamil (`ta_in`), Telugu (`te_in`), Bengali (`bn_in`), and Marathi (`mr_in`).
- **Columns / Features / Labels**:

  - Audio array (16,000 Hz float32 waveform) - Transcription text label
- **Target Variable**: Audio transcription string.
- **Where Used in Code**: Reference benchmark configs and evaluation files in `models/polywhisper/README.md` and `models/polywhisper/baselines_resumable.py`.
- **Model / Task it Supports**: PolyWhisper ASR per-language LoRA adapter training & FLEURS word error rate (WER) benchmarking.
- **Citation**:

```
CODE
```

```
bibtex
  @inproceedings{fleurs2022,
    title={FLEURS: Few-shot Learning Evaluation of Universal Representations of Speech},
    author={Conneau, Alexis and et al.},
    booktitle={IEEE SLT},
    year={2022}
  }
```

- **Privacy / Ethical Notes**: Open CC-BY licensed datasets recorded with contributor consent.